📚 LESSON 12: RANDOM & TIME MODULES

Unlocking Python's Standard Library

Module: Modules & Libraries

Neon City Cyber Academy

Junior Agent Training Protocol

👥 Your Team Today

Emily Chen - Handler

> "Modules are like tool kits. Python comes with dozens of them built-in. Today we unlock randomness and time control!"

Cole Martinez - Tech Lead

> "The import system is fundamental to Python. Understanding modules and namespaces separates beginners from professional developers."

📝 Quick Review: What We've Built

So far, we've used:

- Variables, strings, math
- Conditionals and loops
- Functions

But Python has SO MUCH MORE built-in!

Today: Learn to import and use external code.

🎯 What Are Modules?

Module: A Python file containing functions, classes, and variables you can use.

Python's Standard Library: 200+ modules included with Python!

Examples:

- `random` - generate random numbers
- `time` - work with time and delays
- `math` - advanced math functions
- `datetime` - dates and times
- `json` - work with JSON data

📦 The `import` Statement

Basic syntax:

```
import random

# Now you can use random's functions
number = random.randint(1, 10)
print(number)  # Random number 1-10
```

Pattern: `import module_name`

💬 Emily Explains: Why Modules?

> "Why not include everything by default?
>
> 1. **Faster startup** - only load what you need
> 2. **Namespace organization** - avoid name conflicts
> 3. **Modularity** - mix and match features
>
> If Python loaded EVERYTHING at startup, your 'Hello World' would take 10 seconds to run!"

🎲 The `random` Module

Purpose: Generate random numbers and make random choices.

```python
import random

# Random integer
dice = random.randint(1, 6)
print(f"You rolled: {dice}")

# Random float (0.0 to 1.0)
value = random.random()
print(f"Random value: {value}")
```

🎯 Example: Dice Roll Simulator

```
import random

def roll_dice():
    return random.randint(1, 6)

print("🎲 Rolling dice...")
roll1 = roll_dice()
roll2 = roll_dice()

print(f"Die 1: {roll1}")
print(f"Die 2: {roll2}")
print(f"Total: {roll1 + roll2}")
```

🎮 Challenge #1: Number Guessing Game

Mission: Create a guessing game with random numbers.

```
import random

secret = random.randint(1, 10)
guess = int(input("Guess (1-10): "))

if guess == secret:
    print("🎯 Correct!")
else:
    print(f"❌ Wrong! It was {secret}")
```

Random Choice from List

```python
import random

agents = ["Nova", "Phoenix", "Shadow", "Blade"]

selected = random.choice(agents)
print(f"Selected agent: {selected}")

# Shuffle a list
random.shuffle(agents)
print(f"Shuffled: {agents}")
```

choice() - pick one random element

shuffle() - randomize order (modifies list!)

💬 Cole's Randomness Insight

> "Computers are 100% deterministic - same input = same output.
>
> So how do we get 'random' numbers?
>
> **Pseudorandom Number Generators (PRNGs):**
>
> - Use complex math to generate sequences
> - LOOK random, but are actually deterministic
> - Seeded with current time by default
>
> For cryptography, use `secrets` module (truly random)!"

⏰ The `time` Module

Purpose: Work with time, delays, timestamps.

```
import time

print("Starting...")
time.sleep(2)  # Pause for 2 seconds
print("Done!")
```

`time.sleep(seconds)` – pause program execution

🎯 Example: Countdown Timer

```
import time

for i in range(5, 0, -1):
    print(f"T-minus {i}...")
    time.sleep(1)

print("🚀 Launch!")
```

Output: Prints countdown with 1-second delays!

🕐 Getting Current Time

```python
import time

# Unix timestamp (seconds since Jan 1, 1970)
timestamp = time.time()
print(f"Timestamp: {timestamp}")

# Readable time
readable = time.ctime()
print(f"Current time: {readable}")
```

Example output:

```
Timestamp: 1703875234.567
Current time: Fri Dec 29 14:30:34 2023
```

💬 Emily's Use Cases for time

> "Common uses for `time` module:
>
> 1. **Delays** - `sleep()` for pauses
> 2. **Benchmarking** - measure code execution time
> 3. **Timestamps** - record when events happen
> 4. **Rate limiting** - prevent too-frequent actions
>
> For complex date/time work, use `datetime` module!"

🎮 Challenge #2: Reaction Time Test

Mission: Measure how fast the user can react!

```
import time
import random

print("Press Enter when you see GO!")
time.sleep(random.randint(1, 3))
print("GO!")

start = time.time()
input()  # Wait for Enter
end = time.time()

reaction = end - start
print(f"Reaction time: {reaction:.3f} seconds")
```

🔧 Different Import Styles

Style 1: Import entire module

```
import random
x = random.randint(1, 10)
```

Style 2: Import specific functions

```
from random import randint
x = randint(1, 10)  # No "random." prefix!
```

Style 3: Import with alias

```
import random as rnd
x = rnd.randint(1, 10)
```

💬 Cole's Import Best Practices

> "Import style rules:
>
> ✅ `import module` - safest, clearest
> ✅ `from module import specific_thing` - for frequently used functions
> ❌ `from module import *` - AVOID! Pollutes namespace
>
> **PEP 8 order:**
>
> 1. Standard library imports
> 2. Third-party imports
> 3. Your own modules
>
> Separate groups with blank lines!"

🎯 Example: Password Generator

```
import random
import string

def generate_password(length=12):
    characters = string.ascii_letters + string.digits
    password = ""

    for i in range(length):
        password += random.choice(characters)

    return password

print(generate_password())
print(generate_password(16))
```

🎮 Challenge #3: Rock Paper Scissors

Mission: Create a game against the computer!

```
import random

choices = ["rock", "paper", "scissors"]
computer = random.choice(choices)
player = input("Choose (rock/paper/scissors): ")

print(f"Computer chose: {computer}")

# Your win logic here
```

💡 Solution: Rock Paper Scissors

```
import random

choices = ["rock", "paper", "scissors"]
computer = random.choice(choices)
player = input("Choose (rock/paper/scissors): ").lower()

print(f"Computer chose: {computer}")

if player == computer:
    print("🤝 Tie!")
elif (player == "rock" and computer == "scissors" or
      player == "scissors" and computer == "paper" or
      player == "paper" and computer == "rock"):
    print("🎉 You win!")
else:
    print("💀 Computer wins!")
```

📊 Measuring Code Performance

```python
import time

# Method 1: Loop
start = time.time()
total = 0
for i in range(1000000):
    total += i
end = time.time()
print(f"Loop time: {end - start:.4f}s")

# Method 2: Formula
start = time.time()
total = 999999 * 1000000 // 2
end = time.time()
print(f"Formula time: {end - start:.6f}s")
```

Formula is MUCH faster! This is algorithmic thinking.

💬 Emily's Optimization Lesson

> "That example shows why algorithms matter:
>
> **Loop:** O(n) - 1 million operations
> **Formula:** O(1) - 1 operation
>
> Smart algorithms can make code **thousands of times faster**.
> This is what computer science is all about!"

🌍 Creating Your Own Modules

File: `my_tools.py`

```
def greet(name):
    return f"Hello, {name}!"

def add(a, b):
    return a + b
```

File: `main.py`

```
import my_tools

print(my_tools.greet("Agent"))
print(my_tools.add(5, 3))
```

You can create your own module files!

💬 Cole's Module Organization

> "As projects grow:
>
> ```
> my_project/
>     main.py
>     utils.py
>     game.py
>     config.py
> ```
>
> Break code into logical modules:
>
> - `utils.py` - helper functions
> - `config.py` - settings/constants
> - `game.py` - game logic
>
> Each module has a focused purpose!"

🎯 Example: Simple Animation

```
import time
import random

symbols = ["⠷", "⠯", "⠟", "⠻", "⠽", "⠾", "⠳", "⠞", "⠧", "⠼"]

print("Loading", end="")
for i in range(20):
    print(f"\r Loading {random.choice(symbols)}", end="")
    time.sleep(0.1)
print("\r✅ Complete!    ")
```

Creates a spinning loader animation!

🐛 Common Error: Module Not Found

```
import my_module  # ❌ ModuleNotFoundError
```

Causes:

1. Typo in module name
2. Module not installed (`pip install module_name`)
3. File not in Python's search path

Fix: Check spelling and installation!

🐛 Common Error: Circular Imports

File A imports File B, File B imports File A = 💥

```
# a.py
import b

# b.py
import a  # ❌ Circular!
```

Fix: Restructure your code to avoid cycles!

💬 Emily's Essential Modules to Learn

> "After mastering basics, explore these:
>
> 📊 **math** - advanced math functions
> 📅 **datetime** - dates and times
> 📂 **os** - operating system functions
> 📄 **json** - data serialization
> 🌐 **requests** - HTTP requests (3rd party)
> 🎨 **tkinter** - GUI applications
> 🐍 **pygame** - game development
>
> The standard library is HUGE - explore it!"

🎓 Key Takeaways

- ✅ **Modules** contain reusable code
- ✅ `import` loads modules into your program
- ✅ `random` for randomness (dice, choices, games)
- ✅ `time` for delays and timestamps
- ✅ **Standard library** = 200+ built-in modules
- ✅ You can **create your own modules**
- ✅ Proper **import organization** matters

🚀 Course Progress: Foundations Complete!

You've mastered:

- ✅ Variables, strings, math operators
- ✅ Conditionals (if/elif/else)
- ✅ Loops (while, for, nested)
- ✅ Functions (parameters, returns)
- ✅ Modules (random, time)

Next modules: Lists, dictionaries, file I/O, OOP!

🎯 Final Challenge: Mystery Number

Mission: Combine random, time, and functions!

```
import random
import time

def play_game():
    secret = random.randint(1, 100)
    attempts = 0
    start = time.time()

    # Your game loop here (unlimited guesses)
    # Track attempts and time
    # Give "higher/lower" hints

play_game()
```

💡 Solution: Mystery Number

```
import random
import time

def play_game():
    secret = random.randint(1, 100)
    attempts = 0
    start = time.time()

    print("🎯 Guess the number (1-100)!")

    while True:
        guess = int(input("Your guess: "))
        attempts += 1

        if guess == secret:
            elapsed = time.time() - start
            print(f"✅ Correct in {attempts} tries!")
            print(f"⏱️ Time: {elapsed:.1f}s")
            break
        elif guess < secret:
            print("📈 Higher!")
        else:
            print("📉 Lower!")

play_game()
```

📊 Mission Stats

Concept: Modules (random & time)

Difficulty: ★★★★

Skills Unlocked:

- Import system
- Random number generation
- Time delays & measurement
- Module organization
- Standard library awareness

Status: READY FOR ADVANCED TOPICS! 🎯